Object–oriented programming

Instructor: Masoud Asghari

Web page: http://www.masses.ir/lectures/oops2017sut

Ch: 4

In this slide

- We follow:
  - https://docs.oracle.com/javase/tutorial/index.html
  - **Trail: Learning the Java Language > Classes and Objects**

Classes

This section shows you the anatomy of a class, and how to declare fields, methods, and constructors.

Objects

This section covers creating and using objects. You will learn how to instantiate an object, and, once instantiated, how to use the `dot` operator to access the object's instance variables and methods.

More on Classes

This section covers more aspects of classes that depend on using object references and the `dot` operator that you learned about in the preceding section: returning values from methods, the `this` keyword, class vs. instance members, and access control.

Nested Classes

Static nested classes, inner classes, anonymous inner classes, local classes, and lambda expressions are covered. There is also a discussion on when to use which approach.

Enum Types

This section covers enumerations, specialized classes that allow you to define and use sets of constants.

An Example

```java
public class Bicycle {

    // the Bicycle class has three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int startCadence, int startSpeed, int startGear) {
        gear = startGear; cadence = startCadence;
        speed = startSpeed;
    }

    // the Bicycle class has four methods
    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }

}
```

```java
public class MountainBike extends Bicycle {

    // the MountainBike subclass has
    // one field
    public int seatHeight;

    // the MountainBike subclass has
    // one constructor
    public MountainBike(int startHeight, int startCadence,
                        int startSpeed, int startGear) {
        super(startCadence, startSpeed, startGear);
        seatHeight = startHeight;
    }

    // the MountainBike subclass has
    // one method
    public void setHeight(int newValue) {
        seatHeight = newValue;
    }

}
```

Declaring Classes

```
class MyClass {
    // field, constructor, and
    // method declarations
}
```

- This is a *class declaration*.
  - The class body (the area between the braces)
  - contains all the code that provides for the life cycle of the objects created from the class:
    - constructors for initializing new objects,
    - declarations for the fields that provide the state of the class and its objects,
    - and methods to implement the behavior of the class and its objects.
- You can provide more information about the class,
  - such as the name of its superclass,
  - whether it implements any interfaces,
  - and so on, at the start of the class declaration.

```
class MyClass extends MySuperClass implements YourInterface {
    // field, constructor, and
    // method declarations
}
```

  - You can also add modifiers like public or at the very beginning

Declaring Classes

- In general, class declarations can include these components, in order:
  1. Modifiers such as *public* or no modifier.
  2. The class name,
     - with the initial letter capitalized by convention.
  3. The name of the class's parent (superclass), if any,
     - preceded by the keyword extends.
     - A class can only extend (subclass) one parent.
  4. A comma-separated list of interfaces implemented by the class,
     - if any, preceded by the keyword implements.
     - A class can implement more than one interface.
  5. The class body, surrounded by braces, {}.

Declaring Member Variables

- Member variables in a class—these are called fields.
- Field declarations are composed of three components, in order:
  1. Zero or more modifiers, such as public.
  2. The field's type.
  3. The field's name.
- **Access Modifiers**
  - The first (left-most) modifier lets you control what other classes have access to a member field.
  - public modifier—the field is accessible from all classes.
  - private modifier—the field is accessible only within its own class.
  - In the spirit of encapsulation, it is common to make fields private.
    - Use public methods to access the private variables

Defining Methods

- Method declarations have six components, in order:
  1. Modifiers—such as public, private, .
  2. The return type—the data type of the value returned by the method, or void if the method does not return a value.
  3. The method name
     - By convention, method names should be a **verb in lowercase** or a multi-word name that begins with a verb in lowercase, followed by adjectives, nouns, etc. In multi-word names, the first letter of each of the second and following words should be capitalized.
  4. The parameter list in parenthesis
     - a comma-delimited list of input parameters, preceded by their data types, enclosed by parentheses, ().
     - If there are no parameters, you must use empty parentheses.
  5. An exception list—to be discussed later.
  6. The method body, enclosed between braces { }
     - the method's code, including the declaration of local variables
- The only required elements of a method declaration are the method's return type, name, a pair of parentheses, (), and a body between braces, {}.
- Definition: Two of the components of a method declaration comprise the **method signature**—the method's name and the parameter types

```
calculateAnswer(double, int, double, double)
```

Overloading Methods

- Java can distinguish between methods with different method signatures.
  - methods within a class can have the same name if they have different parameter lists
- Overloaded methods are differentiated by the number and the type of the arguments passed into the method.
- The compiler does not consider return type when differentiating methods, so you cannot declare two methods with the same signature even if they have a different return type.

Providing Constructors for Your Classes

- A class contains constructors that are invoked to create objects from the class blueprint
- Constructor declarations look like method declarations except
  - they use the name of the class , and have no return type.

```
public class Bicycle {

    public int cadence;
    public int gear;
    public int speed;

    public Bicycle(int startCadence, int startSpeed, int startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    public Bicycle() {
        gear = 1;
        cadence = 10;
        speed = 0;
    }

}
```

- To create a new Bicycle object called myBike, a constructor is called by the new operator:

```
Bicycle myBike = new Bicycle(30, 0, 8);
```

- It creates space in memory for the object and initializes its fields.

Providing Constructors for Your Classes

- You can provide no constructor for your class,
  - The compiler automatically provides a no-argument, default constructor for any class without constructors
  - This default constructor will call the no-argument constructor of the superclass.
  - In this situation, the compiler will complain if the superclass doesn't have a no-argument constructor so you must verify that it does.
- If your class has no explicit superclass, then it has an implicit superclass of **Object**, which does have a no-argument constructor.

- You can use access modifiers in a constructor's declaration to control which other classes can call the constructor.
  - Note: If another class cannot call a MyClass constructor, it cannot directly create MyClass objects.

Passing Information to a Method or a Constructor

- The parameters of a method are used in the method body and at runtime will take on the values of the arguments that are passed in.
  - **Note**: *Parameters* refers to the list of variables in a method declaration. *Arguments* are the actual values that are passed in when the method is invoked.
- **Arbitrary Number of Arguments**
  - You can use a construct called varargs to pass an arbitrary number of values to a method.
  - You use varargs when you don't know how many of a particular type of argument will be passed to the method.
  - It's a shortcut to creating an array manually
  - To use varargs, you follow the type of the last parameter by an ellipsis (three dots, ...), then a space, and the parameter name.
  - The method can then be called with any number of that parameter, including none.
  - The method can be called either with an array or with a sequence of arguments.
  - The code in the method body will treat the parameter as an array in either case.

```
public Polygon polygonFrom(Point... corners) {
    int numberOfSides = corners.length;
    double squareOfSide1, lengthOfSide1;
    squareOfSide1 = (corners[1].x - corners[0].x)* (corners[1].x - corners[0].x)
    + (corners[1].y - corners[0].y)* (corners[1].y - corners[0].y);
    lengthOfSide1 = Math.sqrt(squareOfSide1);
}
```

Passing Information to a Method or a Constructor

- You will most commonly see varargs with the printing methods; for example, this printf method:

```
public PrintStream printf(String format, Object... args)
```

- allows you to print an arbitrary number of objects. It can be called like this:

```
System.out.printf("%s: %d, %s%n", name, idnum, address);
```

- or like this

```
System.out.printf("%s: %d, %s, %s, %s%n", name, idnum, address, phone, email);
```

- or with yet a different number of arguments.

- **Parameter Names**
  - A parameter can have the same name as one of the class's fields.
  - In this case, the parameter is said to <u>shadow the field</u>.
  - So using the simple names within the body of the method refers to the parameter, not to the field.
  - To access the field, you must use a qualified name (using **<u>this</u>**)

Passing Arguments

- Primitive arguments, such as an int or a double, are passed into methods by value.
  - This means that any changes to the values of the parameters exist only within the scope of the method. When the method returns, the parameters are gone and any changes to them are lost.
- Reference data type parameters, such as objects, are also passed into methods by value.
  - However, the values of the object's fields can be changed in the method, if they have the proper access level. when the method returns, myCircle still references the same Circle object as before the method was called.

For example, consider a method in an arbitrary class that moves `Circle` objects:

```
public void moveCircle(Circle circle, int deltaX, int deltaY) {
    // code to move origin of circle to x+deltaX, y+deltaY
    circle.setX(circle.getX() + deltaX);
    circle.setY(circle.getY() + deltaY);

    // code to assign a new reference to circle
    circle = new Circle(0, 0);
}
```

Let the method be invoked with these arguments:

```
moveCircle(myCircle, 23, 56)
```

Objects

- A typical Java program creates many objects, Once an object has completed the work for which it was created, its resources are recycled for use by other objects.
- Here's a small program, called CreateObjectDemo, that creates three objects: one Point object and two Rectangle objects.
  - You will need all three source files to compile this program.

```java
public class Point {
    public int x = 0;
    public int y = 0;
    // a constructor!
    public Point(int a, int b) {
    x = a;
    y = b;
    }
}
```

```java
public class Rectangle {
    public int width = 0;
    public int height = 0;
    public Point origin;

    public Rectangle() {
    origin = new Point(0, 0);
    }
    public Rectangle(Point p) {
    origin = p;
    }
    public Rectangle(int w, int h) {
    origin = new Point(0, 0);
    width = w;
    height = h;
    }
    public Rectangle(Point p, int w, int h) {
    origin = p;
    width = w;
    height = h;
    }

    public void move(int x, int y) {
    origin.x = x;
    origin.y = y;
    }

    public int getArea() {
    return width * height;
    }
}
```

```java
public class CreateObjectDemo {

    public static void main(String[] args) {

        // Declare and create a point object and two rectangle objects.
        Point originOne = new Point(23, 94);
        Rectangle rectOne = new Rectangle(originOne, 100, 200);
        Rectangle rectTwo = new Rectangle(50, 100);

        // display rectOne's width, height, and area
        System.out.println("Width of rectOne: " + rectOne.width);
        System.out.println("Height of rectOne: " + rectOne.height);
        System.out.println("Area of rectOne: " + rectOne.getArea());

        // set rectTwo's position
        rectTwo.origin = originOne;

        // display rectTwo's position
        System.out.println("X Position of rectTwo: " + rectTwo.origin.x);
        System.out.println("Y Position of rectTwo: " + rectTwo.origin.y);

        // move rectTwo and display its new position
        rectTwo.move(40, 72);
        System.out.println("X Position of rectTwo: " + rectTwo.origin.x);
        System.out.println("Y Position of rectTwo: " + rectTwo.origin.y);
    }
}
```

Creating Objects

- a class provides the blueprint for objects;
  - you create an object from a class.
  - Example:

```
Point originOne = new Point(23, 94);
```

  - This statement has 3 parts:

1. **Declaration**: The code set in bold are all variable declarations that associate a variable name with an object type.
2. **Instantiation**: The new keyword is a Java operator that creates the object.
3. **Initialization**: The new operator is followed by a call to a constructor, which initializes the new object.

1. Declaring a Variable to Refer to an Object

- to declare a variable, you write:

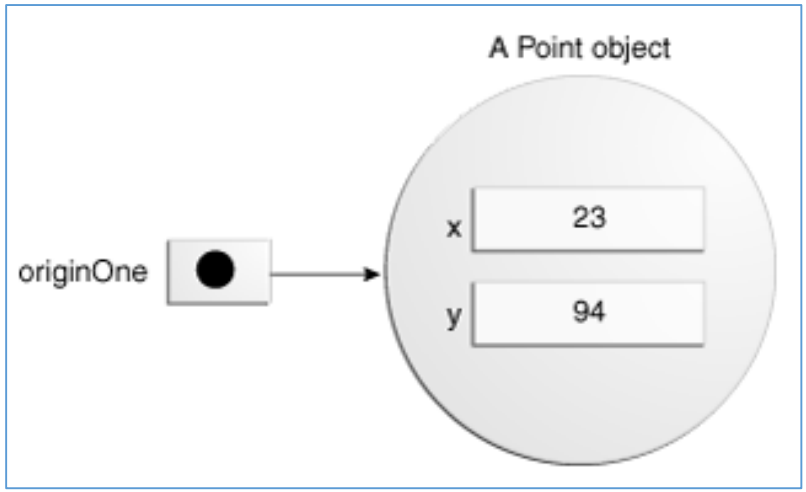
```
type name;
```

- This notifies the compiler that you will use name to refer to data whose type is type.
- With a primitive variable, this declaration also reserves the proper amount of memory for the variable.
- But for a reference variable, its value will be undetermined until an object is actually created and assigned to it.
- Simply declaring a reference variable does not create an object. For that, you need to use the **new** operator,

Instantiating a Class

- The new operator instantiates a class (creates an object of the class)
  - by allocating memory for a new object and returning a reference to that memory.
  - The new operator also invokes the object constructor.
- The new operator requires the name of the class to instantiate.
- The new operator returns a reference to the object it created. This reference is usually assigned to a variable of the appropriate type

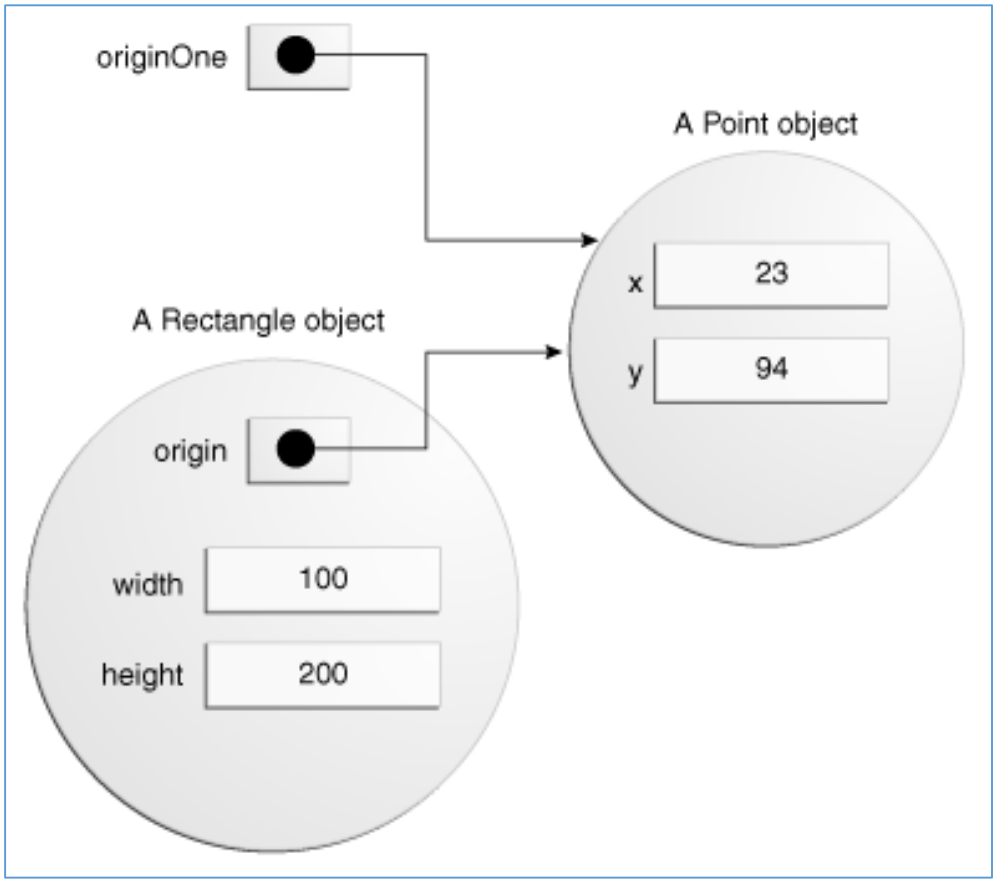
```
Point originOne = new Point(23, 94);
```



- The reference returned by the new operator does not have to be assigned to a variable. It can also be used directly in an expression.

```
int height = new Rectangle().height;
```

```
Point originOne = new Point(23, 94);
Rectangle rectOne = new Rectangle(originOne, 100, 200);
```



Using Objects

- Once you've created an object,
  - you probably want to use it for something.
  - You may need to use the value of one of its fields,
  - change one of its fields,
  - or call one of its methods to perform an action.
- Referencing an Object's Fields
  - Object fields are accessed by their name.
  - Code that is inside the object's class can use a field name, Unless the field name is shadowed by an parameter.
  - Code that is outside the object's class must use an object reference or expression, followed by the dot (.) operator, followed by a simple field name
    - When you access an instance field through an object reference, you reference that particular object's field.
    - To access a field, you can use a named reference to an object,
    - or you can use any expression that returns an object reference.
    - Recall that the new operator returns a reference to an object.
    - So you could use the value returned from **new** to access a new object's fields:

```
int height = new Rectangle().height;
```

    - Note that after this statement has been executed, the program no longer has a reference to the created Rectangle, because the program never stored the reference anywhere. The object is unreferenced, and its resources are free to be recycled by the Java Virtual Machine.

Calling an Object's Methods

- You also use an object reference to invoke an object's method.
  - You append the method's simple name to the object reference, with an intervening dot operator (.).
  - Also, you provide, within enclosing parentheses ( ), any arguments to the method.
  - If the method does not require any arguments, use empty parentheses.

```
Rectangle rectTwo = new Rectangle(50, 100);
rectTwo.move(40, 72);
```

  - Like fields, you could use the value returned from new to invoke a new object's methods

```
new Rectangle(100, 50).getArea()
```

The Garbage Collector

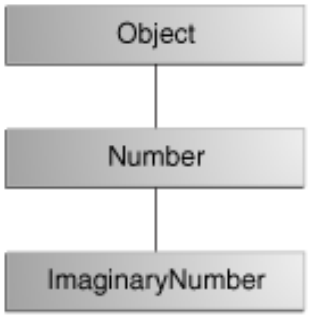
- The Java platform allows you to create as many objects as you want
  - and you don't have to worry about destroying them.
  - The Java runtime environment deletes objects when it determines that they are no longer being used.
  - This process is called ***garbage collection***.
- An object is eligible for garbage collection
  - when there are no more references to that object.
  - References that are held in a variable are usually dropped when the variable goes out of scope.
  - Or, you can explicitly drop an object reference by setting the variable to the special value null.
  - Remember that a program can have multiple references to the same object; all references to an object must be dropped before the object is eligible for garbage collection.

- The Java runtime environment has a garbage collector that periodically frees the memory used by objects that are no longer referenced. The garbage collector does its job automatically when it determines that the time is right.

More on Classes
Returning a Value from a Method

- A method returns to the code that invoked it when it
  - completes all the statements in the method,
  - Or reaches a return statement
  - Or throws an exception
- Any method declared void doesn't return a value.
  - It does not need to contain a return statement, but it may do so.
  - In such a case, a return statement can be used to branch out of a control flow block and exit the method
- Any method that is not declared void
  - must contain a return statement with a corresponding return value
- A method can also return a reference type (object).
  - When a method uses a class name as its return type, the class of the type of the returned object must be either a subclass of, or the exact class of, the return type.
  - Example: The **returnANumber** method can return an **ImaginaryNumber** but not an **Object**.

```
public Number returnANumber() {
    ...
}
```



- **Note: You also can use interface names as return types.**
  In this case, the object returned must implement the specified interface.

Using the this Keyword

- Within an instance method or a constructor,
  - this is a reference to the **current object** (the object whose method or constructor is being called). You can refer to any member of the current object from within an instance method or a constructor by using this.
- Using this with a Field
  - The most common reason for using the **this** keyword is because a field is shadowed by a method or constructor parameter.

```java
public class Point {
    public int x = 0;
    public int y = 0;
    public Point(int a, int b) {
        x = a;
        y = b;
    }
}
```

Using this

```java
public class Point {
    public int x = 0;
    public int y = 0;
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

- From within a constructor, you can also use the **this** keyword to call another constructor in the same class. Doing so is called an *explicit constructor invocation*.
  - **the invocation of another constructor must be the first line in the constructor**

```java
public class Rectangle {
    private int x, y;
    private int width, height;
    public Rectangle() { this(0, 0, 1, 1); }
    public Rectangle(int width, int height) {this(0, 0, width, height); }
    public Rectangle(int x, int y, int width, int height) {
        this.x = x;  this.y = y;  this.width = width;  this.height = height;
    }
}
```

Controlling Access to Members of a Class

- A **class**
  - may be declared with the modifier public, in which case that class is visible to all classes everywhere.
  - If a class has no modifier (the default, also known as *package-private*), it is visible only within its own package.
- At the **member level** (Fields and methods),
  - you can also use the public modifier or no modifier (package-private) just as with top-level classes, and with the same meaning.
  - For members, there are two additional access modifiers: private and protected.
  - The private modifier specifies that the member can only be accessed in its own class.
  - The protected modifier specifies that the member can only be accessed within its own package (as with package-private) and, in addition, by a subclass of its class in another package.
- Access levels affect you in two ways.
  - First, when you use classes that come from another source, such as the classes in the Java platform, access levels determine which members of those classes your own classes can use.
  - Second, when you write a class, you need to decide what access level every member variable and every method in your class should have.

Controlling Access to Members of a Class

Modifier	Class	Package	Subclass	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
no modifier	Y	Y	N	N
`private`	Y	N	N	N

- The first data column indicates whether the class itself has access to the member defined by the access level. As you can see, a class always has access to its own members.
- The second column indicates whether classes in the same package as the class (regardless of their parentage) have access to the member.
- The third column indicates whether subclasses of the class declared outside this package have access to the member.
- The fourth column indicates whether all classes have access to the member.

- **Tips on Choosing an Access Level:**
  - Use the most restrictive access level that makes sense for a particular member. Use private unless you have a good reason not to.
  - Avoid public fields except for constants.
    - Public fields tend to link you to a particular implementation and limit your flexibility in changing your code.

Understanding Class Members

- We discussed the use of the static keyword to create fields and methods that belong to the class, rather than to an instance of the class.
- When a number of objects are created from the same class blueprint, they each have their own distinct copies of **instance variables**. Each of these objects has its own values for these variables, stored in different memory locations.
- Sometimes, you want to have variables that are common to all objects of a particular class.
  - This is accomplished with the static modifier. Fields that have the static modifier in their declaration are called static fields or class variables.
  - They are associated with the class (they have defined), rather than with any object (of that class).
  - Every instance of the class shares that class's variables, which is in one fixed location in memory.
  - Any object (of the class) can change the value of the class variable, but class variables can also be manipulated without creating an instance of the class.
  - Class variables are referenced by the **class name** itself
    - This makes it clear that they are class variables.
    - You can also refer to static fields with an object reference (But it does not make it clear that they are class variables)

Understanding Class Members

- Run-able code:
  - **Note**: Myclass has no-modifier (*package-private*),
    - Otherwise (if public), it should be declared in another file (Myclass.java)
  - **Note:** *test* is public static,
    - it could not be private static , In this case it only could be accessed by the class itself

```
class Myclass{
    public static int test=100;
}

public class JavaApplication1 {
    public static void main(String[] args) {
        Myclass.test++;
        System.out.println("Ctest: "+Myclass.test );//101
        Myclass objMyclass=new Myclass();
        objMyclass.test++;
        System.out.println("Otest: "+objMyclass.test );//102
        System.out.println("Ctest: "+Myclass.test );//102
    }
}
```

Understanding Class Members

- For example,
  - suppose you want to create a number of Bicycle objects and assign each a serial number, beginning with 1 for the first object.
  - This ID number is unique to each object and is therefore an instance variable.
  - At the same time, you need a field to keep track of how many Bicycle objects have been created so that you know what ID to assign to the next one.
  - Such a field is not related to any individual object, but to the class as a whole.
  - For this you need a class variable, numberOfBicycles, as follows:

```
public class Bicycle {

    private int cadence;
    private int gear;
    private int speed;
    private int id;
    private static int numberOfBicycles = 0;

    public Bicycle(int startCadence, int startSpeed, int startGear){
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
        // increment number of Bicycles
        // and assign ID number
        id = ++numberOfBicycles;
    }
}
```

Understanding Class Members

- **Class Methods**
  - Static methods, which have the static modifier in their declarations, should be invoked with the class name, without the need for creating an instance of the class, as in `ClassName.methodName(args)`
  - **Note**: You can also refer to static methods with an object reference of the class as: `instanceName.methodName(args)`
    (But it does not make it clear that they are class methods)
- A common use for static methods
  - is to access static fields.
  - For example,
    - we could add a static method to the Bicycle class to access the numberOfBicycles static field

```
public static int getNumberOfBicycles() {
    return numberOfBicycles;
}
```

Understanding Class Members

- Combinations of instance and class variables and methods:
  - Instance methods can access instance variables and instance methods directly.
  - Instance methods can access class variables and class methods directly.
  - Class methods can access class variables and class methods directly.
  - Class methods cannot access instance variables or instance methods directly (they must use an object reference).
  - Also, class methods cannot use the **this** keyword as there is no instance for this to refer to.
- **Constants**
  - The static modifier, in combination with the final modifier, is also used to define constants. The final modifier indicates that the value of this field cannot change.
  - By convention, the names of constant values are spelled in UPPERCASE letters. If the name is composed of more than one word, the words are separated by an underscore (_)
  - Example:

```
static final double PI = 3.141592653589793;
```

Bicycle.java

```java
public class Bicycle {

    private int cadence;
    private int gear;
    private int speed;

    private int id;

    private static int numberOfBicycles = 0;


    public Bicycle(int startCadence,
                   int startSpeed,
                   int startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;

        id = ++numberOfBicycles;
    }

    public int getID() {
        return id;
    }

    public static int getNumberOfBicycles() {
        return numberOfBicycles;
    }

    public int getCadence() {
        return cadence;
    }

    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public int getGear(){
        return gear;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public int getSpeed() {
        return speed;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }
}
```

Runner app

```
import java.util.Scanner;

public class program {
    private static Scanner myScanner = new Scanner(System.in);

    public static void main(String[] args){
        Bicycle myBike = new Bicycle(0,5,0);

        System.out.println("What is your Name?");
        String yourName = myScanner.nextLine();

        System.out.println("What is your Bike's initial speed?");
        int x = myScanner.nextInt();
        Bicycle yourBike = new Bicycle(0,x,0);


        System.out.println("My Bike ID:"+myBike.getID()+
                " speed:"+myBike.getSpeed());

        System.out.println(yourName+"'s Bike ID:"+yourBike.getID()+
                " speed:"+yourBike.getSpeed());

    }
}
```

Initializing Fields

- You can often provide an initial value for a field in its declaration:
  - this form of initialization has limitations because of its simplicity.
  - If initialization requires some logic (for example, a for loop to fill a complex array), simple assignment is inadequate.
  - Instance variables can be initialized in constructors, other logic can be used.
  - To provide the same capability for **class variables**, Java includes static initialization blocks.
- **A static initialization block**
  - is a normal block of code enclosed in braces, { }, and preceded by the static keyword.

```
static {
    // whatever code is needed for initialization goes here
}
```

  - A class can have any number of static initialization blocks, and they can appear anywhere in the class body.
- There is an alternative to static blocks
  - you can write a private static method:

```
class Whatever {
    public static varType myVar = initializeClassVariable();

    private static varType initializeClassVariable() {

        // initialization code goes here
    }
}
```

Initializing Fields

- **Initializing Instance Members**
  - Normally, you would put code to initialize an instance variable in a **constructor**.
  - There are two alternatives to using a constructor to initialize instance variables: initializer blocks and final methods.
    - **Initializer blocks** for instance variables look just like static initializer blocks, but **without** the static keyword
      - The Java compiler copies initializer blocks into every **constructor**. Therefore, this approach can be used to share a block of code between multiple constructors.
    - Using a **final method** for initializing an instance variable
      - A final method cannot be overridden in a subclass.
      - This is especially useful if subclasses might want to reuse the initialization method. The method is final because calling non-final methods during instance initialization can cause problems.

```
class Whatever {
    private varType myVar = initializeInstanceVariable();

    protected final varType initializeInstanceVariable() {

        // initialization code goes here
    }
}
```

Nested Classes

- The Java programming language allows you to define a class within another class. Such a class is called a *nested class*
  - Nested classes are divided into two categories: static and non-static.
  - Nested classes that are declared static are called static nested classes.
  - Non-static nested classes are called inner classes.

```
class OuterClass {
    ...
    static class StaticNestedClass {
        ...
    }
    class InnerClass {
        ...
    }
}
```

  - A nested class is a member of its enclosing class.
  - Non-static nested classes (inner classes) have access to other members of the enclosing class, even if they are declared private.
  - Static nested classes do not have access to other members of the enclosing class.
  - As a member of the OuterClass, a nested class can be declared private, public, protected, or package private. (Recall that outer classes can only be declared public or package private.)

Nested Classes

- **Why Use Nested Classes?**
  - It is a way of logically grouping classes that are only used in one place
  - It increases encapsulation
    - Consider two top-level classes, A and B, where B needs access to members of A that would otherwise be declared private. By hiding class B within class A, A's members can be declared private and B can access them. In addition, B itself can be hidden from the outside world.
  - It can lead to more readable and maintainable code
- **Local and Anonymous Classes**
  - You can declare an inner class within the body of a method. These classes are known as local classes.
  - You can also declare an inner class within the body of a method without naming the class. These classes are known as anonymous classes.
- **Lambda Expressions**
  - for classes with only one method, even an anonymous class seems a bit excessive and cumbersome. Lambda expressions let you express instances of single-method classes more compactly.

Enum Types

- An enum type
  - is a special data type that enables for a variable to be a set of predefined constants.
  - The variable must be equal to one of the values that have been predefined for it.
  - Examples: compass directions (values of NORTH, SOUTH, EAST, and WEST) and the days of the week.
- You define an enum type by using the enum keyword.
  - For example, you would specify a days-of-the-week enum type as:

```
public enum Day {SUNDAY, MONDAY, TUESDAY, WEDNESDAY,THURSDAY, FRIDAY, SATURDAY }
```

  - how to use the Day enum defined above:

```
Day day;
day= Day.FRIDAY;
if(day == Day.FRIDAY)//run some thing
```

Enum Types

- The enum declaration
  - defines a class (called an enum type).
  - The enum class body can include methods and other fields.
  - The compiler automatically adds some special methods when it creates an enum.
  - For example, they have a static *values* method that returns an array containing all of the values of the enum in the order they are declared. This method is commonly used in combination with the for-each construct to iterate over the values of an enum type.
  - For example,

```
for (Day d : Day.values()) {
    System.out.print(d);
}
```

Exercises

- Write a class with fields and methods resembling a TV (ClassTV)
  - Including ON-OFF, Channel, Volume
  - Includes TV object's serial number (starting from 1000)
  - Write methods where copies state of one TV object to another
    - By writing an class method:
      public static void cpTVState(ClassTV fromTV, ClassTV toTV);
    - By writing an instance method:
      public void cpTVStateFrom(ClassTV fromTV);

- Use the above class within the main class of a program
  - Create 3 TV units (obTV1, obTV2, obTV3)
  - Turn On TV1 and put on Channel 2, volume 15
  - Turn On TV2 and copy the state of TV1 to it using cpTVState
  - Turn On TV3 and copy the state of TV2 to it using cpTVStateFrom
  - Turn Off TV2